PLIER ICI

LICENCE 1 · INFORMATIQUE · ÉVALUATION

SQL — la fiche mémo

Volet 1 · écrire une requête
xseorkly.github.io/sql1

1 La requête complète

```
SELECT   brand, COUNT(*) -- afficher quoi ?
FROM     Sephora         -- quel tableau ?
WHERE    price > 20      -- quelles lignes ?
GROUP BY brand           -- des tas par… ?
HAVING   COUNT(*) >= 2   -- quels tas ?
ORDER BY brand DESC      -- quel ordre ?
LIMIT    10;             -- combien max ?
```

Ordre d'écriture (obligatoire)

`SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT`

« **S**ophie **F**ait **W**hisky, **G**uy **H**abite **O**slo, **L**ise »

Ordre d'exécution

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`

Donc `WHERE` ne peut pas utiliser `COUNT` : les tas n'existent pas encore.

2 La méthode : 6 questions

1. **Qu'est-ce que j'affiche ?** → `SELECT`
2. **Dans quel tableau ?** → `FROM Sephora`
3. **Je garde certains produits ?** → `WHERE`
4. **« par… », « pour chaque… » ?** → `GROUP BY`
5. **Je garde certains groupes ?** → `HAVING`
6. **Trier ? les premiers ?** → `ORDER BY` + `LIMIT`

3 Règles d'écriture

Texte	✓ `'Dior'` ✗ Dior ✗ ‘Dior’
Nombres	✓ `29.5` `100000` ✗ `29,5` `100 000`
Casse	avec `=` : `'Dior'` ≠ `'dior'`
Fin	`;` · aller à la ligne ne change rien

4 Les cases vides : NULL

NULL = « je ne sais pas », ni 0 ni vide.

✓ `price IS NULL` · ✓ `IS NOT NULL` · ✗ `= NULL`

À retenir
Une comparaison avec NULL n'est jamais vraie : la ligne n'est gardée ni par `> 20` ni par `<= 20`. Les calculs ignorent les NULL (sauf `COUNT(*)`). Sephora : `rating = 0` = « pas noté ».

5 Choisir les lignes : WHERE

WHERE regarde **chaque ligne une par une** : condition vraie → gardée, sinon → jetée.

`=` égal	`<>` différent
`>` strictement plus grand	`>=` plus grand **ou égal**
`<` strictement plus petit	`<=` plus petit ou égal
`AND` les **deux** vraies	`OR` **au moins une**

AND passe avant OR → parenthèses !
✓ `WHERE (brand = 'Dior' OR brand = 'Nova') AND price > 25`

6 Trois raccourcis

`BETWEEN 20 AND 40`	de 20 à 40, **bornes comprises**
`IN ('Dior', 'Nova')`	Dior **ou** Nova (liste)
`LIKE '%mascara%'`	contient « mascara »
`LIKE 'Mini%'`	commence par « Mini »
`LIKE '%Set'`	finit par « Set »

`%` = n'importe quoi (même rien) · `_` = 1 caractère · LIKE ignore les majuscules (SQLite).

7 Ranger le résultat

`SELECT DISTINCT brand`	chaque valeur **une fois**
`ORDER BY price`	croissant (ASC par défaut)
`ORDER BY price DESC`	décroissant (**DESC**endre)
`ORDER BY price DESC, name`	2e critère = départage
`ORDER BY … LIMIT 3`	les 3 premiers **après tri**

LIMIT sans ORDER BY = lignes au hasard. `SELECT *` = toutes les colonnes.

8 « Je joue l'ordinateur »

1. **WHERE** : je barre les lignes qui ne passent pas (NULL ⇒ barrée).
2. **GROUP BY** : je fais les tas.
3. **Calculs** dans chaque tas (AVG ÷ nb de valeurs **non NULL**).
4. **HAVING** : je barre les tas qui ne passent pas.
5. **ORDER BY** puis **LIMIT**. J'écris **seulement** le résultat.

Exemple : 8 produits dont 1 prix NULL
`COUNT(*)` = 8 · `COUNT(price)` = 7 · somme 270 → `AVG` = 270 ÷ **7** = 38,57 (et pas ÷ 8).

PLIER ICI

CALCULER · GROUPER · SOUS-REQUÊTES

SQL — la fiche mémo

Volet 2 · aller plus loin
xseorkly.github.io/sql1

9 Compter et calculer

Outil	Calcule
`COUNT(*)`	nombre de **lignes**
`COUNT(price)`	lignes où price est **rempli**
`COUNT(DISTINCT brand)`	valeurs **différentes**
`SUM(x)` · `AVG(x)`	total · moyenne
`MIN(x)` · `MAX(x)`	plus petit · plus grand
`ROUND(x, 2)`	arrondi à 2 décimales
`COUNT(*) AS nombre`	étiquette de colonne

Sans GROUP BY → **une seule ligne**. WHERE filtre **avant** de calculer. Aucune ligne → COUNT = 0, les autres = NULL.

10 GROUP BY et HAVING

```
SELECT brand, COUNT(*) AS nombre
FROM Sephora
WHERE price >= 30      -- trie les PRODUITS
GROUP BY brand         -- un tas par marque
HAVING COUNT(*) >= 2;  -- trie les TAS
```

WHERE ou HAVING ?
Condition sur **un produit** (prix, marque) → `WHERE`
Condition sur **un groupe** (COUNT, AVG…) → `HAVING`

Règle d'or du GROUP BY
Dans le SELECT : la colonne du GROUP BY + des calculs, **rien d'autre**.
✗ `SELECT name, COUNT(*) … GROUP BY brand`

11 Sous-requêtes

La parenthèse est calculée **d'abord**.

```
-- UNE valeur → =  >  <
WHERE price > (SELECT AVG(price)
               FROM Sephora)
WHERE price = (SELECT MAX(price)
               FROM Sephora) -- ex aequo
-- une LISTE → IN
WHERE brand IN (SELECT brand FROM Sephora
  GROUP BY brand HAVING COUNT(*) >= 100)
```

Répéter le filtre dans la parenthèse
`WHERE category = 'Mascara' AND price = (SELECT MIN(price) FROM Sephora WHERE category = 'Mascara')`

12 Niveau + : alias et NOT EXISTS

```
-- plus cher que la moyenne de SA catégorie
SELECT p.name FROM Sephora AS p
WHERE p.price > (SELECT AVG(q.price)
  FROM Sephora AS q
  WHERE q.category = p.category);
-- marques sans AUCUNE édition limitée
SELECT DISTINCT s.brand FROM Sephora AS s
WHERE NOT EXISTS (SELECT 1 FROM Sephora AS p
  WHERE p.brand = s.brand
    AND p.limited_edition = 1);
```

13 De la question au SQL

« seulement », « dont », « qui »	`WHERE`
« les différentes », « sans doublon »	`DISTINCT`
« combien » · « total » · « moyenne »	`COUNT` · `SUM` · `AVG`
« le plus / le moins cher »	`MAX` / `MIN`
« par marque », « pour chaque »	`GROUP BY`
« les marques qui ont au moins… »	`GROUP BY` + `HAVING`
« plus cher que la moyenne »	sous-requête
« le top 5 », « les 10 premiers »	`ORDER BY … DESC LIMIT`
« inconnu », « non renseigné »	`IS NULL`
« aucun produit qui… »	`NOT EXISTS`

! Les 8 pièges

1. ‘Dior’ → `'Dior'`	5. COUNT dans WHERE → HAVING
2. `100 000` → `100000`	6. AND + OR sans parenthèses
3. virgule avant FROM	7. LIMIT sans ORDER BY
4. `= NULL` → `IS NULL`	8. colonne hors GROUP BY

✓ Colonnes de Sephora

name · brand · category · price · love · rating (0 = pas noté) · number_of_reviews · value_price · online_only, exclusive, limited_edition (1 = oui, 0 = non). Texte exact : `'CLINIQUE'`, `'tarte'`.

Avant de rendre : **;** partout · ordre S F W G H O L · bonnes colonnes · NULL et ordre vérifiés

PLIER ICI